Phase 29: Technical BA Interview Question Bank

Document Code: TBA-BNK-001

Project: Duplicate Bin Ecosystem

Scope: SQL, Databases, REST APIs, WebSockets, Python, Qt, Packaging & Security

Structure: Question | Short Answer | Detailed Answer | Project Example | Likely Follow-up

1. SQL, Relational Modeling & Database Architecture

Q1: "How does the SQLite persistent cache handle concurrent reads and writes from multiple worker threads without locking errors?"

- **Short Answer:** By enabling SQLite's Write-Ahead Logging (WAL) mode (`PRAGMA journal_mode=WAL`), which decouples readers from writers and prevents `database is locked` exceptions.
- **Detailed Answer:** In standard rollback journal mode, an SQLite write operation locks the entire database file, causing read queries on other threads to fail or block. In our `PersistentHashCache` implementation (`duplicate_bin/engine/cache.py`), we explicitly initialize the database with WAL mode. In WAL mode, changes are appended to a separate `-wal` file, allowing multi-threaded worker pools to read cached hashes concurrently while writer threads record new hash entries without contention.
- **Project Example:** duplicate_bin/engine/cache.py, docs/TECHNICAL_ARCHITECTURE.md#L95-L98.
- **Likely Follow-up:** "*What query would you write to identify the top 5 largest duplicate groups in the database?*"
- *Answer:*

```
sql SELECT path_scanned, duplicates_count, bytes_recoverable FROM scans WHERE status = 'completed' ORDER BY bytes_recoverable DESC LIMIT 5;
```

2. REST APIs, WebSockets & Real-Time Orchestration

Q2: "How does the backend architecture handle real-time progress updates from desktop agents to web browser dashboards?"

- **Short Answer:** Through a WebSocket connection manager that acts as an in-memory pub/sub relay between agent daemons (`/ws/agent`) and manager browser dashboards (`/ws/dashboard`).
- **Detailed Answer:** In `backend/app/websocket_manager.py`, our `ConnectionManager` class maintains two active registries: `active_agents: Dict[device_id, WebSocket]` and `active_dashboards: Dict[user_id, List[WebSocket]]`. When a desktop daemon processes files, it sends JSON frames of type `AGENT_PROGRESS`. The server extracts the `device_id`, resolves the owning `user_id` from the database, and broadcasts the event across all open dashboard WebSockets for that user, updating the browser UI in real time.
- **Project Example:** backend/app/websocket_manager.py, backend/app/main.py#L324-L393.
- **Likely Follow-up:** *"What happens if the network connection between the agent and WebSocket server drops mid-scan?"*
- *Answer: "The WebSocket connection manager catches `WebSocketDisconnect` and removes the agent from `active_agents`. The desktop agent continues its local scan uninterrupted because scanning is purely local. Upon completion, the agent attempts to reconnect and posts its final summary via REST to `/api/scans/telemetry`."*

3. Postman & API Verification

Q3: "How do you test and verify your FastAPI endpoints using Postman or automated test clients?"

- **Short Answer:** We use environment variables for Bearer JWT tokens, write pre-request and test assertion scripts in Postman, and validate schemas using automated `httpx` and `fastapi.testclient.TestClient` test scripts.
- **Detailed Answer:** In our test harness (`tests/verify_saas_integration.py` and `backend/verify_backend.py`), we automate the full test suite:
  1. `POST /api/auth/register` creates an isolated test user.
  2. `POST /api/auth/login` asserts HTTP 200 and extracts `access_token` into the authorization header.
  3. `GET /api/devices/pairing-code` retrieves the 6-digit code.
  4. `POST /api/devices/pair` submits the code and asserts that a 64-character token is returned.
  5. We assert that repeated submissions of the same code return HTTP 400 (anti-replay check).
- **Project Example:** backend/verify_backend.py, tests/verify_saas_integration.py.
- **Likely Follow-up:** *"How do you test WebSocket endpoints in Postman?"*

- *Answer: "Postman supports native WebSocket requests. We configure `ws://127.0.0.1:8000/ws/agent?device_id=UUID&token=HEX64`, connect, and send simulated `AGENT_PROGRESS` JSON frames, observing the broadcast on a second Postman WebSocket connected to `/ws/dashboard`."*

4. Authentication, Authorization & Security Architecture

Q4: "How does the platform prevent path traversal attacks during file archiving operations?"

- **Short Answer:** By resolving canonical absolute file paths using `os.path.realpath` and verifying that the destination path remains strictly within the designated quarantine sandbox directory.
- **Detailed Answer:** If user input or symbolic links contain relative traversal vectors (e.g., `../../../../etc/passwd`), a naive file copy could overwrite critical system files. In `duplicate_bin/engine/archive.py#L42`, our `_safe_archive_path()` function resolves the real physical path of both the source and the quarantine directory. It performs a boundary check: `target_path.startswith(sandbox_root)`. If the path escapes the sandbox, the operation is aborted with a `SecurityError`.
- **Project Example:** docs/TECHNICAL_ARCHITECTURE.md#L141-L142.
- **Likely Follow-up:** *"How do you prevent symlink loops during directory traversal?"*
- *Answer: "Our file collector `collect_files()` explicitly tracks visited device and inode numbers (`st_dev`, `st_ino`) to detect and break circular symbolic link loops."*

5. Python, Concurrency & PySide6 Architecture

Q5: "How did you ensure that long-running file scans and hash calculations did not freeze the PySide6 desktop GUI?"

- **Short Answer:** By decoupling heavy computational workloads onto dedicated worker threads (`QThread`) that communicate with the main GUI loop strictly through Qt Signals and Slots.
- **Detailed Answer:** In GUI programming, running intensive CPU or disk I/O operations on the main thread blocks the operating system event loop, resulting in a frozen window and `(Not Responding)` alerts. In `duplicate_bin/ui/scan_worker.py`, we encapsulate the detection engine inside a `QThread`. The worker thread emits signals: `progress_updated(int)`, `status_message(str)`, and `scan_completed(object)`. The main window connects these signals to UI progress bars and tree

`scan_completed(object)`. The main window connects these signals to UI progress bars and tree widgets, ensuring 60 FPS smooth rendering throughout the entire scan.

- **Project Example:** duplicate_bin/ui/scan_worker.py, docs/TECHNICAL_ARCHITECTURE.md#L104-L106.
- **Likely Follow-up:** *"Can worker threads directly call widget methods like `progressBar.setValue()`?"*
- *Answer: "No. Qt widgets are not thread-safe. Calling widget methods directly from background threads can cause memory corruption or segmentation faults. Worker threads must always communicate via Qt Signals."*

6. Packaging, Build Automation & Distribution

Q6: "How is Duplicate Bin packaged for cross-platform distribution without requiring users to install Python?"

- **Short Answer:** We use PyInstaller for standalone Windows executables (`DuplicateBin.exe`) and native Debian packaging scripts (`build_debian.sh`) for Linux `.deb` packages.
- **Detailed Answer:**
- On **Windows**: `DuplicateBin.spec` configures PyInstaller in `--windowed --onefile` mode, bundling the Python interpreter, PySide6 Qt binaries, and assets into a standalone executable.
- On **Linux**: `build_debian.sh` structures a standard Debian package layout (`debian/control`, `debian/rules`, `/usr/bin/duplicate-bin`, and `/usr/share/applications/duplicate-bin.desktop`), producing an optimized **187 KB** Debian `.deb` package.
- **Project Evidence:** DuplicateBin.spec, build_windows.py, build_debian.sh.
- **Likely Follow-up:** *"How do you handle missing dynamic DLLs or Qt platform plugins during PyInstaller bundling?"*
- *Answer: "In `DuplicateBin.spec`, we explicitly declare `binaries` and `datas` hooks for PySide6 plugins (e.g., `platforms/qwindows.dll`) and bundled theme assets, ensuring all runtime shared libraries are packaged into the executable directory."*